Optimization in Machine Learning (contd.)

Avi Mohan

Backpropagation in three acts

- All the optimization algorithms discussed until now required $\nabla\mathcal{L}(y, \hat{y})$.
  - The NN's weights are contained in $\mathcal{L}$ through the end-to-end function

$$\hat{y} = g(\boldsymbol{x}, \boldsymbol{w})$$

- $g(\cdot, \boldsymbol{w})$ for a deep neural network with several hidden layers, will have a very complex form.
  - Computing partial derivatives directly is computationally very hard and time-consuming.

- Solution: iterative algorithm that computes partial derivatives layer-by-layer.
  - Output layer → Hidden Layers → Input Layer

Preparing to Backprop

- Chain rule 1

$$y = h\big(g(x)\big) \Rightarrow \frac{dy}{dx} = h'\big(g(x)\big)g'(x)$$

- Chain rule 2

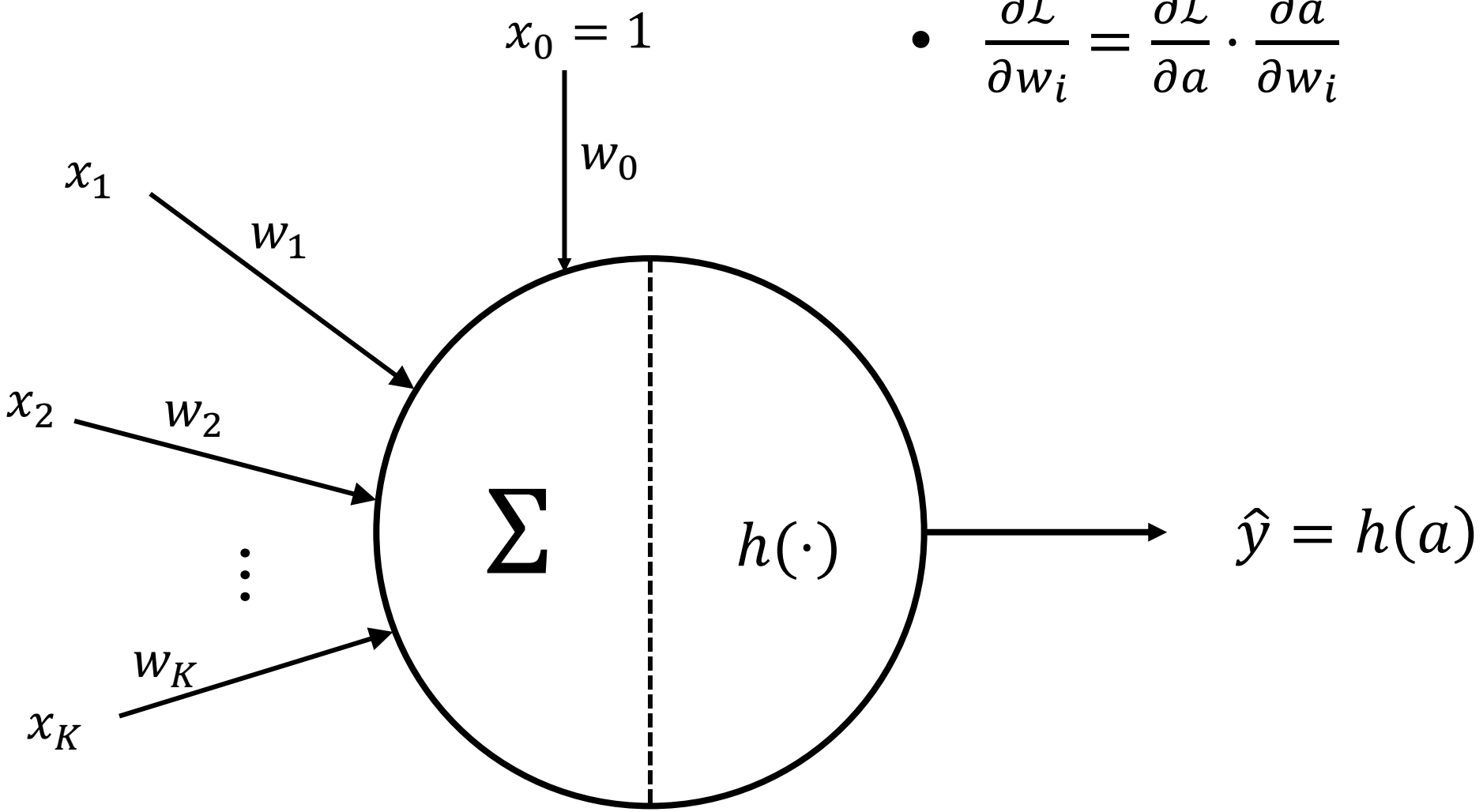
$$z = f(a), a = g(x, y)$$
$$\Rightarrow \frac{\partial z}{\partial x} = f'(a)\frac{\partial a}{\partial x}, \frac{\partial z}{\partial y} = f'(a)\frac{\partial a}{\partial y}$$

- Chain rule 3

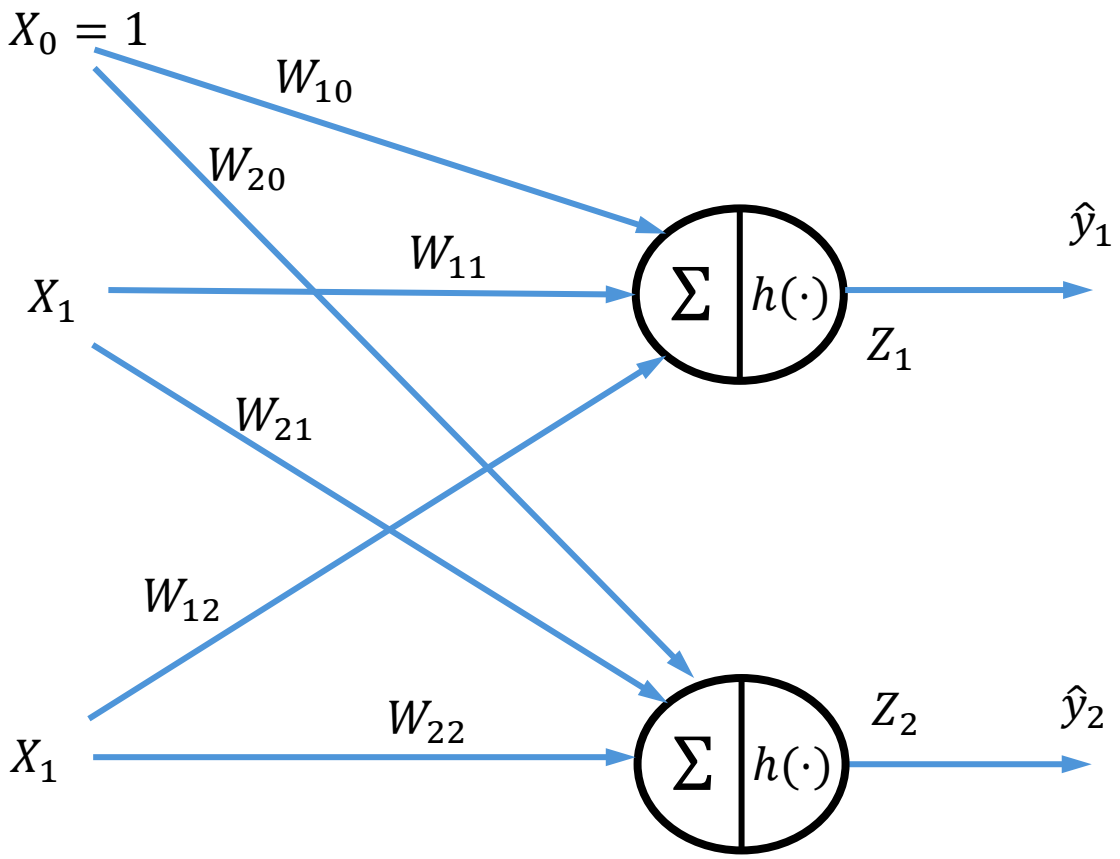
$$z = f(a_1, a_2), a_1 = g(x, y), a_2 = h(x, y)$$
$$\Rightarrow \frac{\partial z}{\partial x} = \frac{\partial z}{\partial a_1}\frac{\partial a_1}{\partial x} + \frac{\partial z}{\partial a_2}\frac{\partial a_2}{\partial x}$$
$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial a_1}\frac{\partial a_1}{\partial y} + \frac{\partial z}{\partial a_2}\frac{\partial a_2}{\partial y}$$

Backpropagation: Act 1

- $a := \sum_{i=0}^{K} w_i x_i$
- $\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial w_i}$



Backpropagation: Act 2



- $a_j := \sum_{i=0}^{K} w_{ji} x_i$
- $\frac{\partial \mathcal{L}}{\partial w_{ji}} = \frac{\partial \mathcal{L}}{\partial a_j} \cdot \frac{\partial a_j}{\partial w_{ji}}$

Backpropagation: Act 3